

DIGITAL NOTICEBOARD

Detailed Project Documentation & Demonstration Guide

Project type	React + Vite web application with Vercel serverless API
Hosted application	https://digitalnoticeboard-one.vercel.app
Repository	GitHub — DigitalNoticeboard
Prepared for	Website Design Deliverable — T2 2026
Documentation scope	Based on the uploaded Digital Noticeboard v5 project source

This document explains the purpose, structure, authentication credentials, roles and permissions, student accounts, news submission workflow, moderation system, API behaviour, data storage, interface features, deployment model, testing procedure, and important technical limitations of the submitted project.

Table of Contents

- | | |
|--------------------------------------|--|
| 1. Project Overview | 10. News Data Structure |
| 2. Application Goals | 11. API Endpoints |
| 3. Technology Stack & Architecture | 12. Data Storage & Vercel Deployment |
| 4. Login Credentials & Test Accounts | 13. Theme, Navigation & UX Details |
| 5. User Roles & Permissions | 14. Security & Important Technical Notes |
| 6. How the Application Works | 15. Testing / Demonstration Checklist |
| 7. Student Workflow | 16. Current Seed Data |
| 8. Admin Workflow & Moderation | 17. Recommended Future Improvements |

9. Pages & Interface Features

SECTION 1

Project Overview

Digital Noticeboard is an online bulletin-board application designed to provide a central place where students and school administrators can publish, review and read school-related announcements. The system replaces a purely physical noticeboard with a searchable web interface and introduces a moderation workflow so that submitted news is reviewed before it becomes visible in the public Daily News feed.

The application is divided into two broad experiences. Students can sign in using a registered student ID, browse approved announcements, search and filter news, and submit new announcements for review. Administrators have access to the same bulletin-board features plus a separate administrative area where submissions can be viewed by status. The Main Admin has the additional authority to approve, reject or permanently delete submissions.

Hosted application: <https://digitalnoticeboard-one.vercel.app>

Project repository: the source code is stored in the GitHub repository named *Digitalnoticeboard* under the account used for the project.

SECTION 2

Application Goals

- Create a simple and modern online bulletin board for a school environment.
- Make publishing announcements easier than using a physical noticeboard.
- Prevent unreviewed submissions from appearing in the public feed.
- Give administrators a clear moderation interface.
- Allow users to find announcements quickly using title search and grade filtering.
- Provide responsive navigation, mobile sidebar controls and a light/dark mode.
- Keep the frontend and API under one Vercel deployment so the hosted project can be accessed from one URL.

Core concept: students submit school news through a structured form. Every new submission is automatically created with a **pending** status on the server. Administrators can review all submissions and see their current moderation status. Only approved news is shown on the Daily News feed. Main Admin can approve, reject or delete submissions. The interface supports light and dark themes. The frontend communicates with the backend through the **/api/notices** API.

SECTION 3

Technology Stack & Architecture

Technology	Purpose
React 18	Builds the user interface from reusable components and pages.
Vite 5	Frontend development server and production build tooling.
React Router 6	Handles routes such as /daily-news, /submit and admin pages.
Axios	Makes HTTP requests from the React frontend to the API.
Node.js + Express	Implements the serverless API handlers under /api.
Vercel	Hosts the frontend and serverless API functions.
Vercel KV / Redis	Current storage layer used by api/_lib/store.js for notice persistence.
Lucide React	Provides interface icons such as search, logout, shield, file and status actions.
CSS custom properties	Controls the application's light/dark colour tokens and theme switching.

High-level architecture

```
Browser
  |
  v
React + Vite frontend
  | Axios requests
  v
/api/notices and /api/notices/:id
  |
  v
Express serverless functions on Vercel
  |
  v
api/_lib/store.js
  |
  v
Vercel KV / Redis · key: notices
```

The application uses React Context for three cross-cutting concerns: authentication (AuthContext), news state and API operations (NewsContext), and theme preference (ThemeContext). This keeps the pages relatively focused on presentation and user interactions.

Important source folders

Path	Responsibility
src/pages/	Main screens: login, Daily News, Submit News and admin pages.
src/components/	Reusable UI: sidebar, topbar, news cards, route protection, modal and breadcrumbs.

Path	Responsibility
src/context/AuthContext.jsx	Student/admin account matching, login state, logout and role flags.
src/context/NewsContext.jsx	Fetches notices, submits notices, updates status and deletes notices.
src/context/ThemeContext.jsx	Light/dark theme state persisted in localStorage.
api/notices.js	List and create notice endpoints.
api/notices/[id].js	Read, update and delete an individual notice.
api/_lib/store.js	Persistence abstraction used by the API.
data/notices.json	Seed data used to initialise the notice store when empty.

SECTION 4

Login Credentials & Test Accounts

The authentication in this version is intentionally a demonstration/prototype system. Credentials are hardcoded in `src/context/AuthContext.jsx`; there is no external user database or real password-authentication service.

Student accounts

Student ID	Student Name	Password
STU-2024-001	Chidi Okonkwo	No password
STU-2024-002	Amaka Nwosu	No password
STU-2024-003	Emeka Adeleke	No password
STU-2024-004	Fatima Suleiman	No password

Student login rule: enter a valid student ID only. Matching is case-insensitive and surrounding whitespace is trimmed. If the ID is not in the hardcoded list, the application displays “Student ID not recognized.”

Admin accounts

Admin Name	Shared Password	Permission Level
Mrs. Adaeze Okoro	DigitalNoticeBoard	Admin — view only
Mr. Tunde Bello	DigitalNoticeBoard	Admin — view only
Main Admin	DigitalNoticeBoard	Main Admin — view, approve, reject, delete

Admin login rule: the typed name must match one of the three configured admin names (case-insensitive, with surrounding whitespace ignored), and the password must exactly match **DigitalNoticeBoard**.

Recommended demonstration order

- 1 Use STU-2024-001 to demonstrate student access.
- 2 Submit a new announcement and observe that it does not immediately appear in Daily News because it starts as pending.
- 3 Log out and use Mrs. Adaeze Okoro with password DigitalNoticeBoard to demonstrate read-only admin access.
- 4 Log out and use Main Admin with password DigitalNoticeBoard to demonstrate moderation actions.
- 5 Approve the test submission and return to Daily News to show that approved news becomes visible.

Security warning

These credentials are suitable for a course/demo submission, not for a real production school system. Because the usernames, student IDs and shared admin password are present in frontend source code, anyone who can inspect the deployed JavaScript can discover them. The document includes the credentials because they are required for demonstrating the submitted prototype.

SECTION 5

User Roles & Permissions

Capability	Student	Regular Admin	Main Admin
Sign in	✓	✓	✓
View approved Daily News	✓	✓	✓
Search news by title	✓	✓	✓
Filter by grade	✓	✓	✓
Submit news	✓	✓	✓
Open All Submitted News	—	✓	✓
Open News by Status	—	✓	✓
See pending/rejected items	—	✓	✓
Approve submission	—	—	✓
Reject submission	—	—	✓
Delete submission	—	—	✓

The distinction between regular admins and Main Admin is implemented through the **isMainAdmin** flag. All administrators satisfy **isAdmin**, so they can enter the admin pages. However, AdminAllNews only renders approval, rejection and delete controls when **isMainAdmin** is true.

Route protection

- Unauthenticated users are redirected to the main login portal.
- Any authenticated user can access Daily News and Submit Your News.
- Students attempting to access an admin-only route are redirected back to Daily News.
- The sidebar hides the entire Admin Access section from students.
- Regular admins can inspect submissions but receive a read-only view.
- Main Admin sees the action buttons for moderation.

SECTION 6

How the Application Works

Step 1 — Login portal

The root route (/) displays an Online Bulletin Board login portal with two choices: Student or Admin. The Student option opens a student ID form, while Admin opens a name/password form. The password field has an eye icon that toggles whether the typed password is masked.

Step 2 — Session state

After successful authentication, the application stores a small user object in browser localStorage under noticeboard_user. This means a refresh can restore the current session without asking the user to sign in again. Logging out removes this localStorage entry.

Step 3 — Loading notices

NewsContext requests `/api/notices` using Axios. The server returns notices sorted by their `dateModified` value, with the newest modified record first. The frontend refreshes the notice list every 10 seconds using a polling interval.

Step 4 — Public visibility

Daily News does not display every stored record. It first filters the API result to only notices whose status is approved. It then applies the optional grade and title-search filters. This is the central moderation rule.

Step 5 — Submission

A user fills in Title, Name, Date, News Text, Grade and optionally an Image. The selected date is converted from the browser's YYYY-MM-DD input format to DD-MM-YYYY for display. The frontend sends the record to the POST API.

Step 6 — Server-side moderation default

The POST endpoint deliberately overwrites the incoming status with pending. This prevents a client from simply sending `status=approved` and making its own announcement public.

Step 7 — Admin review

Administrators see all stored submissions, including pending, approved and rejected items. Main Admin can change the status or delete the item. Regular admins can inspect but cannot perform those actions through the interface.

Step 8 — Public publication

Once a submission becomes approved, the next data refresh makes it eligible for Daily News. The NewsCard displays the title, optional image, contributor name, date, news text and status.

SECTION 7

Student Workflow

A. Sign in

- 1 Open the deployed application.
- 2 Select Student.
- 3 Enter one of the configured IDs, for example STU-2024-001.
- 4 Click Sign In.
- 5 The user is taken to Daily News.

B. Read Daily News

- Only approved announcements are shown.
- The search box searches titles.
- The Filter control supports All Grades, First Grade and Second Grade.
- Each announcement is presented as a card.
- If an image was attached, the image is displayed; otherwise an image-placeholder area is used.
- The card shows the contributor, date, news text and moderation status.

C. Submit an announcement

- 1 Open Submit your news from the sidebar.
- 2 Enter a title.
- 3 Enter the contributor name. The field is pre-filled from the signed-in account but remains editable.
- 4 Choose a date.
- 5 Write the news text.
- 6 Choose First Grade or Second Grade.
- 7 Optionally attach an image.
- 8 Click Submit for Review.
- 9 The interface confirms that the submission has been sent for admin review.

Note

The submission does not bypass moderation. The backend creates it with status pending even if a malicious or modified client attempts to send a different status.

SECTION 8

Admin Workflow & Moderation

A. Admin access

After choosing Admin on the login portal, enter one of the configured names and the shared password. All three administrators can enter the admin section, but only Main Admin receives moderation controls.

B. All Submitted News

This page presents a table containing Title, Submitted By, Date, Grade and Status. For Main Admin, an Actions column also appears. The available actions depend on the current status: an approved record no longer needs an Approve button, while a rejected record no longer needs a Reject button.

C. Approve

Main Admin selects the check/approve control. The frontend immediately updates the visible status optimistically, then sends a PATCH request to the notice endpoint. If the server update fails, the previous state is restored.

D. Reject

Main Admin selects the reject control. The same optimistic-update pattern is used, but the status becomes rejected. Rejected news is not eligible for Daily News because Daily News filters for approved records only.

E. Delete

Main Admin selects the trash/delete control. A confirmation modal appears before the DELETE request is sent. The record is removed from the current list immediately; if the server delete fails, NewsContext re-fetches the list.

F. News by Status

The News by Status page groups all records into Pending, Approved and Rejected sections and shows a count for each group. This provides a quick moderation overview without requiring the admin to inspect the full table.

Moderation lifecycle



SECTION 9

Pages & Interface Features

Route	Page	Purpose
/	Landing	Select Student/Admin login and authenticate.
/daily-news	Daily News	Display approved announcements with search and grade filter.
/submit	Submit Your News	Create a new announcement for moderation.
/admin/all-news	All Submitted News	Admin table for inspecting and, for Main Admin, moderating submissions.
/admin/by-status	News by Status	Group pending, approved and rejected records with counts.

Reusable interface components

- **Sidebar:** navigation, collapsible desktop rail, mobile open/close behaviour and role-based Admin Access visibility.
- **Topbar:** current user name, role indicator, theme switch and logout.
- **NewsCard:** announcement presentation including optional image and status pill.
- **ProtectedRoute:** prevents unauthenticated access and blocks students from admin-only routes.
- **ConfirmModal:** confirmation step for permanent deletion.
- **Breadcrumb:** provides contextual page navigation.

Search and filtering

Daily News uses a title search input and a grade dropdown. Search matching is case-insensitive and checks the title text. Grade filtering supports All Grades, First Grade and Second Grade.

Responsive navigation

The sidebar has separate desktop and mobile behaviours. On desktop it can collapse into an icon-only rail. On smaller screens a menu button opens the sidebar as an overlay, with a backdrop that can be clicked to close it.

SECTION 10

News Data Structure

Field	Example	Meaning
id	1	Unique numeric identifier generated by the API.
title	Midterm Exam Timetable Released	Announcement title.
image	null / data URL	Optional uploaded image stored as a base64 data URL.
name	Mrs. Adaeze Okoro	Displayed contributor name.
date	12-08-2026	Human-readable announcement date.
newsText	The midterm examination...	Main body of the announcement.
grade	First Grade	Grade/category used by the Daily News filter.

Field	Example	Meaning
status	approved	Moderation state: pending, approved or rejected.
dateModified	2026-08-12T09:15:00.000Z	Server timestamp used for newest-first sorting.

Example record

```
{
  "id": 7,
  "title": "New School Announcement",
  "image": null,
  "name": "Chidi Okonkwo",
  "date": "15-08-2026",
  "newsText": "Announcement details go here.",
  "grade": "First Grade",
  "status": "pending",
  "dateModified": "2026-08-15T00:00:00.000Z"
}
```

Image handling note: the current Submit News implementation reads an uploaded image with `FileReader` and stores the result directly as a base64 data URL in the notice object. This is convenient for a prototype but increases record size and is not the preferred approach for a production application.

SECTION 11

API Endpoints

Method	Endpoint	Purpose
GET	/api/notices	Return all notices, newest modified first.
POST	/api/notices	Create a notice. Server assigns an ID, timestamp and pending status.
GET	/api/notices/:id	Return one notice by ID.
PATCH	/api/notices/:id	Merge supplied fields into an existing notice; used for moderation status changes.
DELETE	/api/notices/:id	Permanently remove a notice.

POST behaviour

The server reads all existing notices, calculates the next numeric ID, constructs the new record, forces status: pending, stamps dateModified with the current ISO timestamp, stores the record, and returns HTTP 201.

PATCH behaviour

The dynamic endpoint extracts the final URL segment as the ID, finds the matching record, merges the request body over the existing object, saves the updated collection and returns HTTP 200. The frontend currently uses this primarily to set status to approved or rejected.

DELETE behaviour

The API locates the notice, removes it from the collection, writes the new collection and returns the deleted object. A 404 response is returned when the notice does not exist.

Error handling

- Frontend displays an API error when the news endpoint cannot be reached.
- The submission form shows an error state if required fields are missing or the request fails.
- Delete failures trigger a fresh news fetch to restore the UI.
- Status-update failures revert the optimistic local change.

SECTION 12

Data Storage & Vercel Deployment

The current source code uses `@vercel/kv` in `api/lib/store.js`. The store reads and writes the entire notice collection under the Redis/KV key `notices`. If that key is empty on the first read, the application seeds it from `data/notices.json`.

Deployment flow

- 1 The project source is pushed to GitHub.
- 2 The GitHub repository is imported into Vercel.
- 3 Vercel builds the Vite frontend.
- 4 The `/api` files are deployed as serverless functions.
- 5 The rewrite configuration sends normal frontend routes to `index.html` so React Router can handle them.

- 6 API routes remain under /api and are handled by the serverless functions.
- 7 The frontend uses a relative API path by default, so the same deployed domain serves both frontend and API.

Environment configuration

The source includes an `.env.example` explaining that `VITE_API_URL` can normally remain empty because the frontend and API share the same origin. The uploaded archive also contained a `.env.local` file with a Vercel OIDC token. That token is a credential and is intentionally not reproduced in this documentation.

Credential exposure note

If a credential such as a Vercel token is ever exposed publicly, it should be revoked/rotated immediately. The project's `.gitignore` correctly includes environment-file patterns so `.env.local` should not be committed to Git.

Important correction to older project notes

The README text in the uploaded source describes an older flat-file storage approach and says that Vercel uses `/tmp`. However, the actual current `api/_lib/store.js` in the uploaded project imports `@vercel/kv` and calls `kv.get/kv.set`. Therefore, this documentation treats the current source implementation as authoritative: the active storage abstraction is Vercel KV/Redis, with `data/notices.json` acting as seed data.

Theme, Navigation & UX Details

Light / dark mode

ThemeContext stores the current theme in localStorage under **noticeboard_theme**. The selected theme is applied by adding or removing the **dark** class on the document's root HTML element. CSS custom properties in index.css provide the colour tokens.

Persistent browser state

Storage key	Purpose
noticeboard_user	Keeps the signed-in student/admin object across refreshes.
noticeboard_theme	Keeps the selected light/dark theme across refreshes.

Automatic news refresh

NewsContext fetches news when it mounts and then polls the API every 10 seconds. This means an announcement approved by an admin can appear for another user without manually refreshing the entire page.

Optimistic moderation UI

When Main Admin approves or rejects a notice, the frontend changes the displayed status immediately before waiting for the API. If the API request fails, the previous state is restored. Delete follows the same principle and re-fetches data if the server operation fails.

Visual status language

- **Pending:** awaiting admin review.
- **Approved:** accepted and eligible for Daily News.
- **Rejected:** not eligible for Daily News.
- Status colours are reused across the NewsCard, All Submitted News and News by Status screens for consistency.

Security & Important Technical Notes

What is protected

- React route guards prevent unauthenticated users from navigating directly to protected pages.
- Students are redirected away from admin-only routes.
- The POST API forces new notices to pending status.
- The UI hides moderation actions from non-Main-Admin users.

What is not production-secure

- Authentication is implemented entirely in the frontend with hardcoded accounts.
- The shared admin password is visible in frontend source code.
- Student IDs are also visible in frontend source code.
- API endpoints do not implement server-side user authentication/authorisation in the current source.

- The API PATCH endpoint accepts merged fields generally; a production API should validate which fields each role is allowed to change.
- Uploaded images are stored as base64 strings rather than object-storage URLs.

Recommended production security model

- 1 Move users and roles to a real database.
- 2 Use a real authentication provider or secure server-side sessions/JWTs.
- 3 Never expose admin passwords in frontend code.
- 4 Perform authorisation checks inside the API, not only in React.
- 5 Validate and sanitise submitted fields server-side.
- 6 Limit upload file size and validate image MIME types.
- 7 Store images in object storage and save only their URLs in the database.
- 8 Add audit logs for approval, rejection and deletion.
- 9 Use individual admin passwords rather than one shared password.
- 10 Add rate limiting and abuse protection to public submission endpoints.

Credential handling note

The password DigitalNoticeBoard is documented here because it is part of the course/demo test setup. It should not be reused as a real school administrator password.

SECTION 15

Testing / Demonstration Checklist

Login tests

- ☐ Student login with STU-2024-001 succeeds.
- ☐ Student login with an invalid ID shows an error.
- ☐ Admin login with Main Admin + DigitalNoticeBoard succeeds.
- ☐ Admin login with an incorrect password fails.
- ☐ Admin name matching ignores case and surrounding spaces.
- ☐ Logout returns to the login portal.

Student tests

- ☐ Daily News displays approved seed announcements.
- ☐ Search by title changes the visible results.
- ☐ Grade filter works for First Grade and Second Grade.
- ☐ Submit Your News validates required fields.
- ☐ A new submission reports success and is created as pending.
- ☐ Optional image upload is accepted.

Admin tests

- ☐ Regular admin can open All Submitted News.
- ☐ Regular admin sees a read-only banner and no action buttons.
- ☐ Main Admin sees Approve, Reject and Delete controls.
- ☐ Approving a pending item changes it to approved.
- ☐ Rejecting an item changes it to rejected.
- ☐ Deleting requires confirmation.
- ☐ News by Status shows correct counts and groups.
- ☐ Students cannot reach admin pages by typing their URLs directly.

Persistence / deployment tests

- [] Refreshing the page keeps the signed-in user until logout.
- [] Refreshing the page keeps the chosen theme.
- [] The API responds from the deployed /api/notices route.
- [] A newly approved item becomes visible in Daily News after the next data refresh.
- [] Verify Vercel KV is connected in the deployed environment before relying on data persistence.

SECTION 16

Current Seed Data

ID	Title	Contributor	Grade	Status
1	Midterm Exam Timetable Released	Mrs. Adaeze Okoro	First Grade	approved
2	Inter-House Sports Day — Save the Date	Mr. Tunde Bello	Second Grade	approved
3	Library Extended Hours During Exam Week	Admin Office	First Grade	approved
4	Cafeteria Closed for Maintenance This Friday	Facilities Team	Second Grade	approved
5	Lost Water Bottle — Blue with Stickers	Chinedu Eze	First Grade	rejected
6	Unofficial Bake Sale Announcement	Grace Adamu	Second Grade	rejected

What this means on first load

- Daily News initially has four approved announcements available to display.
- The rejected records remain visible to administrators but are excluded from Daily News.
- There are no pending seed records, so a fresh admin status screen can show Pending (0) until a new submission is created.
- A newly submitted notice will enter the pending group.

SECTION 17

Recommended Future Improvements

The current application is a strong functional course/demo prototype. The following improvements would make it more suitable for real institutional use:

Priority	Improvement	Reason
High	Real authentication + server-side authorisation	Prevents users from bypassing frontend-only role checks.
High	Individual admin accounts	Removes the shared password and improves accountability.
High	Database-backed users and notices	Provides reliable, structured persistence and user management.
High	API validation	Prevents unauthorised field changes and malformed records.
Medium	Object storage for images	Avoids large base64 records and improves performance.
Medium	Admin audit log	Records who approved, rejected or deleted each notice.
Medium	Pagination	Keeps large admin lists performant.
Medium	Notice categories	Allows filtering beyond grade.
Low	Notifications	Could notify students when important announcements are published.

Priority	Improvement	Reason
Low	Rich text editor	Would allow formatted announcement content.
Low	Analytics dashboard	Could show publication counts, pending workload and engagement.

Final project summary

Digital Noticeboard demonstrates a complete frontend-to-server workflow: role-based entry, protected navigation, structured content submission, moderation states, administrative review, searchable public content, persistent theme preferences, API communication and deployment through Vercel. The most important demonstration feature is the moderation pipeline: a submission is never published directly; it enters pending status, is reviewed by an administrator, and only approved content reaches Daily News.

Primary demonstration credentials

Main Admin — password: DigitalNoticeBoard | Student example — STU-2024-001 (no password)

End of documentation.